

第 7 讲：时序差分方法

第 7 讲：时序差分方法

本章介绍强化学习中的时序差分（Temporal-Difference, TD）方法。与蒙特卡洛（MC）学习类似，TD 学习也是免模型的（model-free），但由于其增量形式而具有一些优势。在第 6 章的准备下，读者看到 TD 学习算法时将不会感到突兀。事实上，TD 学习算法可以视为求解贝尔曼方程或贝尔曼最优方程的特殊随机逼近算法。

由于本章将介绍不少 TD 算法，我们首先概述这些算法并阐明它们之间的关系：

- 第 7.1 节介绍最基本的 TD 算法，用于估计给定策略的状态价值。在理解其他 TD 算法之前，先理解这个基本算法非常重要。
- 第 7.2 节介绍 Sarsa 算法，它可以估计给定策略的动作价值。该算法可以结合策略改进步骤来寻找最优策略。Sarsa 算法可以很自然地通过将第 7.1 节中状态价值估计替换为动作价值估计得到。
- 第 7.3 节介绍 n 步 Sarsa 算法，它是 Sarsa 算法的推广。将展示 Sarsa 和 MC 学习是 n 步 Sarsa 的两个特例。
- 第 7.4 节介绍 Q-learning 算法，这是最经典的强化学习算法之一。其他 TD 算法旨在求解给定策略的贝尔曼方程，而 Q-learning 旨在直接求解贝尔曼最优方程以获得最优策略。
- 第 7.5 节比较本章介绍的各种 TD 算法，并提供一个统一的视角。

7.1 状态价值的 TD 学习

TD 学习通常指一类广泛的强化学习算法。例如，本章介绍的所有算法都属于 TD 学习的范畴。然而，本节中的 TD 学习特指一种用于估计状态价值的经典算法。

7.1.1 算法描述

给定策略 π ，我们的目标是估计所有 $s \in \mathcal{S}$ 的 $v_\pi(s)$ 。假设我们有一些遵循 π 生成的经验样本 $(s_0, r_1, s_1, \dots, s_t, r_{t+1}, s_{t+1}, \dots)$ 。这里 t 表示时间步。以下 TD 算法可以利用这些样本来估计状态价值：

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t) [v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1}))], \quad (1)$$

$$v_{t+1}(s) = v_t(s), \quad \forall s \neq s_t, \quad (2)$$

其中 $t = 0, 1, 2, \dots$ 。这里， $v_t(s_t)$ 是在时刻 t 对 $v_\pi(s_t)$ 的估计； $\alpha_t(s_t)$ 是在时刻 t 状态 s_t 的学习率。

需要注意的是，在时刻 t ，只有被访问到的状态 s_t 的值被更新，未访问到的状态 $s \neq s_t$ 的值保持不变，如式(2)所示。式(2)通常为了简洁而被省略，但应牢记在心，因为缺少该式算法在数学上是不完整的。

第一次看到 TD 学习算法的读者可能会好奇为什么它被设计成这样。事实上，它可以视为求解贝尔曼方程的一种特殊随机逼近算法。为此，首先回忆状态价值的定义：

$$v_{\pi}(s) = \mathbb{E}[R_{t+1} + \gamma G_{t+1} \mid S_t = s], \quad s \in \mathcal{S}.$$

可以将上式改写为:

$$v_{\pi}(s) = \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s], \quad s \in \mathcal{S}.$$

这是因为 $\mathbb{E}[G_{t+1} \mid S_t = s] = \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s, a) v_{\pi}(s') = \mathbb{E}[v_{\pi}(S_{t+1}) \mid S_t = s]$ 。上式是贝尔曼方程的另一种表达形式, 有时被称为**贝尔曼期望方程**。

TD 算法可以通过应用 Robbins-Monro 算法 (第 6 章) 求解上述贝尔曼期望方程来推导。感兴趣的读者可以在下面的推导框中查看细节。

证明

TD 算法的推导

下面展示式(1)中的 TD 算法可以通过应用 Robbins-Monro 算法求解贝尔曼期望方程得到。

对于状态 s_t , 定义函数:

$$g(v_{\pi}(s_t)) := v_{\pi}(s_t) - \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s_t].$$

那么, 贝尔曼期望方程等价于 $g(v_{\pi}(s_t)) = 0$ 。

由于我们可以获得 r_{t+1} 和 s_{t+1} , 它们分别是 R_{t+1} 和 S_{t+1} 的样本, 因此我们可以得到的 $g(v_{\pi}(s_t))$ 的带噪观测为:

$$\tilde{g}(v_{\pi}(s_t)) = v_{\pi}(s_t) - (r_{t+1} + \gamma v_{\pi}(s_{t+1})) = \underbrace{(v_{\pi}(s_t) - \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s_t])}_{g(v_{\pi}(s_t))} + \underbrace{(\mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s_t] - (r_{t+1} + \gamma v_{\pi}(s_{t+1})))}_{\eta}$$

因此, 用于求解 $g(v_{\pi}(s_t)) = 0$ 的 Robbins-Monro 算法 (第 6.2 节) 为:

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t) \tilde{g}(v_t(s_t)) = v_t(s_t) - \alpha_t(s_t) (v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1}))),$$

其中 $v_t(s_t)$ 是在时刻 t 对 $v_{\pi}(s_t)$ 的估计, $\alpha_t(s_t)$ 是学习率。

该算法与式(1)拥有相似的表达式。唯一的区别是右端包含 $v_{\pi}(s_{t+1})$, 而式(1)包含 $v_t(s_{t+1})$ 。这是因为该算法仅用于在假设其他状态的状态价值已经已知的情况下估计 s_t 的状态价值。如果我们想要估计所有状态的状态价值, 那么右端的 $v_{\pi}(s_{t+1})$ 应该替换为 $v_t(s_{t+1})$, 这就得到了式(1)。这种替换是否仍能保证收敛? 答案是肯定的, 这将在定理 7.1 中得到证明。

7.1.2 性质分析

我们更仔细地审视 TD 算法的表达式。式(1)可以描述为:

$$\underbrace{v_{t+1}(s_t)}_{\text{新估计}} = \underbrace{v_t(s_t)}_{\text{当前估计}} - \alpha_t(s_t) \underbrace{(v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1})))}_{\text{TD 误差 } \delta_t},$$

其中

$$\bar{v}_t := r_{t+1} + \gamma v_t(s_{t+1})$$

称为 **TD 目标**, 而

$$\delta_t := v_t(s_t) - \bar{v}_t = v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1}))$$

称为 **TD 误差**。可以看出, 新估计 $v_{t+1}(s_t)$ 是当前估计 $v_t(s_t)$ 与 TD 误差 δ_t 的组合。

- 为什么 $\bar{v}_t$ 被称为 TD 目标？

这是因为 $\bar{v}_t$ 是算法试图将 $v(s_t)$ 驱动到的目标值。从式(1)两端减去 $\bar{v}_t$ 得到：

$$v_{t+1}(s_t) - \bar{v}_t = (v_t(s_t) - \bar{v}_t) - \alpha_t(s_t)(v_t(s_t) - \bar{v}_t) = [1 - \alpha_t(s_t)](v_t(s_t) - \bar{v}_t).$$

两边取绝对值得到：

$$|v_{t+1}(s_t) - \bar{v}_t| = |1 - \alpha_t(s_t)| |v_t(s_t) - \bar{v}_t|.$$

由于 $\alpha_t(s_t)$ 是一个小的正数，我们有 $0 < 1 - \alpha_t(s_t) < 1$ ，因此：

$$|v_{t+1}(s_t) - \bar{v}_t| < |v_t(s_t) - \bar{v}_t|.$$

这表明新值 $v_{t+1}(s_t)$ 比旧值 $v_t(s_t)$ 更接近 $\bar{v}_t$ 。因此，该算法在数学上将 $v_t(s_t)$ 驱向 $\bar{v}_t$ 。这就是 $\bar{v}_t$ 被称为 TD 目标的原因。

- TD 误差的解释是什么？

该误差被称为时序差分 (temporal-difference)，因为 $\delta_t = v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1}))$ 反映了两个时间步 t 和 $t+1$ 之间的差异。其次，当状态价值估计准确时，TD 误差在期望意义下为零。当 $v_t = v_\pi$ 时：

$$\mathbb{E}[\delta_t | S_t = s_t] = \mathbb{E}[v_\pi(S_t) - (R_{t+1} + \gamma v_\pi(S_{t+1})) | S_t = s_t] = 0.$$

因此，TD 误差反映了估计 v_t 与真实状态价值 v_π 之间的差异。

在更抽象的层面上，TD 误差可以解释为**新息 (innovation)**，它表示从经验样本中获得的新信息。TD 学习的基本思想是：基于新获得的信息修正对状态价值的当前估计。

第二，式(1)中的 TD 算法只能估计给定策略的状态价值。要寻找最优策略，还需要进一步计算动作价值并进行策略改进，这将在第 7.2 节中介绍。

第三，TD 学习与 MC 学习都是免模型的，其优缺点总结在表 1 中。

表 1: TD 学习与 MC 学习的比较

TD 学习	MC 学习
增量式： 收到经验样本后立即更新状态/动作价值。	非增量式： 必须等待回合完全收集完毕才能计算折扣回报。
可处理持续性任务： 增量形式支持分幕式和持续性任务。	仅处理分幕式任务： 只能在回合终止后计算回报。
自助法 (Bootstrapping)： 更新依赖先前估计，需要初始猜测。	非自助法： 直接估计价值，无需初始猜测。
低估计方差： 涉及更少的随机变量。	高估计方差： 涉及许多随机变量，如 $R_{t+1}, R_{t+2}, \dots$

7.1.3 收敛性分析

定理 7.1 TD 学习的收敛性

给定策略 π ，通过式(1)中的 TD 算法， $v_t(s)$ 在 $t \rightarrow \infty$ 时以概率 1 收敛到 $v_\pi(s)$ (对所有 $s \in \mathcal{S}$)，如果满足 $\sum_t \alpha_t(s) = \infty$ 且 $\sum_t \alpha_t^2(s) < \infty$ (对所有 $s \in \mathcal{S}$)。

关于 α_t ：条件 $\sum_t \alpha_t(s) = \infty$ 和 $\sum_t \alpha_t^2(s) < \infty$ 必须对所有 $s \in \mathcal{S}$ 成立。注意在时刻 t ，如果 s 被访问则 $\alpha_t(s) > 0$ ，否则 $\alpha_t(s) = 0$ 。条件 $\sum_t \alpha_t(s) = \infty$ 要求状态 s 被访问无限次。这要

求要么满足探索起点条件，要么使用探索性策略。在实际中，学习率通常选为一个小正常数；此时 $\sum_t \alpha_t^2(s) < \infty$ 不再成立，但可以证明算法在期望意义下收敛。

证明

定理 7.1 的证明

我们基于第 6 章的定理 6.3 证明收敛性。考虑任意状态 $s \in \mathcal{S}$ 。在时刻 t ，由 TD 算法有：

$$v_{t+1}(s) = v_t(s) - \alpha_t(s) \left(v_t(s) - (r_{t+1} + \gamma v_t(s_{t+1})) \right), \quad \text{若 } s = s_t,$$

或 $v_{t+1}(s) = v_t(s)$ (若 $s \neq s_t$)。

定义估计误差为 $\Delta_t(s) := v_t(s) - v_\pi(s)$ 。从 $s = s_t$ 情况减去 $v_\pi(s)$ ：

$$\Delta_{t+1}(s) = (1 - \alpha_t(s))\Delta_t(s) + \alpha_t(s)(r_{t+1} + \gamma v_t(s_{t+1}) - v_\pi(s)), \quad s = s_t.$$

从 $s \neq s_t$ 情况减去 $v_\pi(s)$ 得 $\Delta_{t+1}(s) = \Delta_t(s) = (1 - \alpha_t(s))\Delta_t(s) + \alpha_t(s) \cdot 0$ 。

因此，无论 $s = s_t$ 与否，统一表达式为：

$$\Delta_{t+1}(s) = (1 - \alpha_t(s))\Delta_t(s) + \alpha_t(s)\eta_t(s),$$

其中 $\eta_t(s) = r_{t+1} + \gamma v_t(s_{t+1}) - v_\pi(s)$ (当 $s = s_t$) 或 $\eta_t(s) = 0$ (当 $s \neq s_t$)。这就是定理 6.3 中的过程。

第一个条件在定理 7.1 中已假设成立。对于第二个条件 $\|\mathbb{E}[\eta_t(s) \mid \mathcal{H}_t]\|_\infty \leq \gamma \|\Delta_t(s)\|_\infty$ ：由马尔可夫性质， $\mathbb{E}[\eta_t(s) \mid \mathcal{H}_t] = \mathbb{E}[\eta_t(s)]$ 。对于 $s \neq s_t$ ， $\eta_t(s) = 0$ ，平凡成立。对于 $s = s_t$ ：

$$\begin{aligned} \mathbb{E}[\eta_t(s)] &= \mathbb{E}[r_{t+1} + \gamma v_t(s_{t+1}) - v_\pi(s_t) \mid s_t] \\ &= \mathbb{E}[r_{t+1} + \gamma v_t(s_{t+1}) \mid s_t] - v_\pi(s_t) \\ &= \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s_t) [v_t(s') - v_\pi(s')]. \end{aligned}$$

于是 $|\mathbb{E}[\eta_t(s)]| \leq \gamma \sum_{s'} p(s' \mid s_t) \max_{s'} |v_t(s') - v_\pi(s')| = \gamma \|\Delta_t(s)\|_\infty$ 。第三个条件由于 r_{t+1} 有界也成立。因此过程收敛。以上证明受 [32] 启发。

7.2 动作价值的 TD 学习：Sarsa

第 7.1 节介绍的 TD 算法只能估计状态价值。本节介绍 **Sarsa** 算法，它可以直接估计动作价值，并可结合策略改进步骤来学习最优策略。

7.2.1 算法描述

给定策略 π ，假设我们有以下遵循 π 生成的经验样本： $(s_0, a_0, r_1, s_1, a_1, \dots, s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \dots)$ 。Sarsa 算法为：

$$\begin{aligned} q_{t+1}(s_t, a_t) &= q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - (r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})) \right], \\ q_{t+1}(s, a) &= q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t), \end{aligned} \quad (3)$$

其中 $t = 0, 1, 2, \dots$ 。这里， $q_t(s_t, a_t)$ 是对 $q_\pi(s_t, a_t)$ 的估计， $\alpha_t(s_t, a_t)$ 是学习率。在时刻 t ，只有 (s_t, a_t) 的 q 值被更新。

关于 Sarsa 算法的重要性质：

- 为什么叫“Sarsa”? 因为每次迭代需要 $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$ 。Sarsa 是 state-action-reward-state-action 的缩写。最早在 [35] 中提出, 名称由 [3] 创造。
- Sarsa 为什么被设计成这样? Sarsa 与 TD 算法类似, 可以通过将状态价值估计替换为动作价值估计很自然地得到。
- Sarsa 在数学上做了什么? 与 TD 算法类似, Sarsa 是求解给定策略的贝尔曼方程的随机逼近算法:

$$q_{\pi}(s, a) = \mathbb{E}[R + \gamma q_{\pi}(S', A') \mid s, a], \quad \forall (s, a).$$

这是用动作价值表示的贝尔曼方程。

证明

证明上式是贝尔曼方程

如第 2.8.2 节所述, 用动作价值表示的贝尔曼方程为:

$$q_{\pi}(s, a) = \sum_r r p(r \mid s, a) + \gamma \sum_{s'} \sum_{a'} q_{\pi}(s', a') p(s' \mid s, a) \pi(a' \mid s').$$

由于 $p(s', a' \mid s, a) = p(s' \mid s, a) p(a' \mid s') := p(s' \mid s, a) \pi(a' \mid s')$ (由条件独立性), 上式可重写为:

$$q_{\pi}(s, a) = \sum_r r p(r \mid s, a) + \gamma \sum_{s'} \sum_{a'} q_{\pi}(s', a') p(s', a' \mid s, a).$$

根据期望的定义, 上式等价于 $q_{\pi}(s, a) = \mathbb{E}[R + \gamma q_{\pi}(S', A') \mid s, a]$ 。因此, 这就是贝尔曼方程。

定理 7.2 Sarsa 的收敛性

给定策略 π , 通过式(3)中的 Sarsa 算法, $q_t(s, a)$ 在 $t \rightarrow \infty$ 时以概率 1 收敛到动作价值 $q_{\pi}(s, a)$ (对所有 (s, a)), 如果满足 $\sum_t \alpha_t(s, a) = \infty$ 且 $\sum_t \alpha_t^2(s, a) < \infty$ (对所有 (s, a))。

证明与定理 7.1 类似, 此处省略。

7.2.2 通过 Sarsa 学习最优策略

式(3)中的 Sarsa 只能估计给定策略的动作价值。要寻找最优策略, 需将其与策略改进步骤结合。这种组合通常也统称为 Sarsa, 实现过程见算法。

算法 7.1：通过 Sarsa 学习最优策略

初始化： $\alpha_t(s, a) = \alpha > 0$ 对所有 (s, a) 和所有 t 。 $\epsilon \in (0, 1)$ 。初始化 $q_0(s, a)$ 对所有 (s, a) 。由 q_0 导出初始 ϵ -贪心策略 π_0 。

目标： 学习能从初始状态 s_0 引导智能体到达目标状态的最优路径。

对于每个回合：

1. 在 s_0 处遵循 $\pi_0(s_0)$ 生成 a_0
2. 如果 s_t ($t = 0, 1, 2, \dots$) 不是目标状态：
 - (a) 给定 (s_t, a_t) 收集经验样本 $(r_{t+1}, s_{t+1}, a_{t+1})$ ：通过与环境交互生成 r_{t+1}, s_{t+1} ；遵循 $\pi_t(s_{t+1})$ 生成 a_{t+1} 。
 - (b) 更新 q 值：

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) [q_t(s_t, a_t) - (r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1}))]$$

- (c) 更新 s_t 处的 ϵ -贪心策略：

$$\pi_{t+1}(a | s_t) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s_t)|} (|\mathcal{A}(s_t)| - 1), & a = \arg \max_a q_{t+1}(s_t, a) \\ \frac{\epsilon}{|\mathcal{A}(s_t)|}, & \text{否则} \end{cases}$$

- (d) $s_t \leftarrow s_{t+1}, a_t \leftarrow a_{t+1}$

每次迭代有两个步骤：第一步更新被访问的 q 值，第二步将策略更新为 ϵ -贪心策略。 q 值更新仅针对被访问的单个状态-动作对。策略更新基于**广义策略迭代**思想：策略被立即更新并用于生成下一个样本。策略是 ϵ -贪心的以保持探索性。

仿真示例（图 7.2）演示了 Sarsa 算法。任务是从特定起始状态到目标状态寻找最优路径。奖励设置： $r_{\text{target}} = 0$, $r_{\text{forbidden}} = r_{\text{boundary}} = -10$, $r_{\text{other}} = -1$ 。 $\alpha = 0.1$, $\epsilon = 0.1$, 初始 $q_0(s, a) = 0$, 初始策略均匀分布 $\pi_0(a | s) = 0.2$ 。最终策略能成功引导到目标，但某些状态的策略可能非最优（未被充分探索）。总奖励逐步增加，回合长度逐步减少。 ϵ -贪心策略可能导致偶尔采取非最优动作，使用衰减 ϵ 可以缓解此问题。

证明

Expected Sarsa

Expected Sarsa 是 Sarsa 的变体：

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) [q_t(s_t, a_t) - (r_{t+1} + \gamma \mathbb{E}[q_t(s_{t+1}, A)])],$$

其中 $\mathbb{E}[q_t(s_{t+1}, A)] = \sum_a \pi_t(a | s_{t+1}) q_t(s_{t+1}, a) := v_t(s_{t+1})$ 。Expected Sarsa 与 Sarsa 的不同仅在于 TD 目标：Expected Sarsa 使用 $r_{t+1} + \gamma \mathbb{E}[q_t(s_{t+1}, A)]$ ，而 Sarsa 使用 $r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$ 。虽然计算期望值略微增加复杂度，但减少了估计方差（随机变量从 $\{s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}\}$ 减少到 $\{s_t, a_t, r_{t+1}, s_{t+1}\}$ ）。

Expected Sarsa 可视为求解以下方程的随机逼近算法：

$$q_\pi(s, a) = \mathbb{E} [R_{t+1} + \gamma \mathbb{E}[q_\pi(S_{t+1}, A_{t+1}) | S_{t+1}] \mid S_t = s, A_t = a].$$

代入 $\mathbb{E}[q_\pi(S_{t+1}, A_{t+1}) | S_{t+1}] = v_\pi(S_{t+1})$ 即得贝尔曼方程。更多细节见 [3, 36, 37]。

7.3 动作价值的 TD 学习: n 步 Sarsa

本节介绍 n 步 Sarsa, 它是 Sarsa 的扩展。Sarsa 和 MC 学习是 n 步 Sarsa 的两个极端特例。回忆动作价值 $q_\pi(s, a) = \mathbb{E}[G_t \mid S_t = s, A_t = a]$, 其中 $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$ 。 G_t 可以分解为不同形式:

$$\begin{aligned} \text{Sarsa} &\longleftrightarrow G_t^{(1)} = R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}); \\ G_t^{(2)} &= R_{t+1} + \gamma R_{t+2} + \gamma^2 q_\pi(S_{t+2}, A_{t+2}); \\ &\vdots \\ n \text{ 步 Sarsa} &\longleftrightarrow G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^n q_\pi(S_{t+n}, A_{t+n}); \\ &\vdots \\ \text{MC} &\longleftrightarrow G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \end{aligned}$$

注意 $G_t = G_t^{(1)} = G_t^{(2)} = G_t^{(n)} = G_t^{(\infty)}$, 上标仅表示不同分解结构。

将不同 $G_t^{(n)}$ 代入 $q_\pi(s, a)$ 得到不同算法:

- $n = 1$: $q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid s, a]$, 对应 Sarsa 算法(3)。
- $n = \infty$: $q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \dots \mid s, a]$, 对应 MC 学习: $q_{t+1}(s_t, a_t) = g_t := r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$ 。
- 一般 n : 对应 n 步 Sarsa:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - (r_{t+1} + \gamma r_{t+2} + \dots + \gamma^n q_t(s_{t+n}, a_{t+n})) \right]. \quad (4)$$

要实现 n 步 Sarsa, 需要等到时刻 $t+n$ 才能更新 (s_t, a_t) 的 q 值。式(4)可重写为:

$$\begin{aligned} q_{t+n}(s_t, a_t) &= q_{t+n-1}(s_t, a_t) - \alpha_{t+n-1}(s_t, a_t) \left[q_{t+n-1}(s_t, a_t) \right. \\ &\quad \left. - (r_{t+1} + \gamma r_{t+2} + \dots + \gamma^n q_{t+n-1}(s_{t+n}, a_{t+n})) \right]. \end{aligned}$$

n 步 Sarsa 将 Sarsa 和 MC 作为两个极端特例, 其性能介于两者之间: n 较大时接近 MC (高方差、小偏差), n 较小时接近 Sarsa (大偏差、低方差)。这里介绍的 n 步 Sarsa 仅用于策略评估, 需与策略改进结合以学习最优策略。更多信息见 [3, 第 7 章]。

7.4 最优动作价值的 TD 学习: Q-learning

本节介绍 **Q-learning 算法**, 这是最经典的强化学习算法之一 [38, 39]。Sarsa 只能估计给定策略的动作价值, 而 Q-learning 可以直接估计最优动作价值并找到最优策略。

7.4.1 算法描述

Q-learning 算法为:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - (r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} q_t(s_{t+1}, a)) \right], \quad (5)$$

$$q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t),$$

其中 $t = 0, 1, 2, \dots$ 。 $q_t(s_t, a_t)$ 是对 (s_t, a_t) 的最优动作价值的估计。

Q-learning 与 Sarsa 仅在 TD 目标上有所不同: Q-learning 的 TD 目标是 $r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)$, 而 Sarsa 的是 $r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$ 。此外, 给定 (s_t, a_t) , Sarsa 需要 $(r_{t+1}, s_{t+1}, a_{t+1})$, 而 Q-learning 仅需 (r_{t+1}, s_{t+1}) 。

Q-learning 是求解以下贝尔曼最优方程的随机逼近算法：

$$q(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_a q(S_{t+1}, a) \mid S_t = s, A_t = a \right].$$

证明

证明上式是贝尔曼最优方程

根据期望的定义，该方程可重写为：

$$q(s, a) = \sum_r p(r \mid s, a) r + \gamma \sum_{s'} p(s' \mid s, a) \max_{a \in \mathcal{A}(s')} q(s', a).$$

对等式两端取最大值得到：

$$\max_{a \in \mathcal{A}(s)} q(s, a) = \max_{a \in \mathcal{A}(s)} \left[\sum_r p(r \mid s, a) r + \gamma \sum_{s'} p(s' \mid s, a) \max_{a \in \mathcal{A}(s')} q(s', a) \right].$$

记 $v(s) := \max_{a \in \mathcal{A}(s)} q(s, a)$ ，上式变为：

$$v(s) = \max_{a \in \mathcal{A}(s)} \left[\sum_r p(r \mid s, a) r + \gamma \sum_{s'} p(s' \mid s, a) v(s') \right],$$

这正是第 3 章介绍的贝尔曼最优方程。Q-learning 的收敛性分析与定理 7.1 类似，见 [32, 39]。

7.4.2 离线策略 (Off-policy) vs 在线策略 (On-policy)

使 Q-learning 略为特殊的是：Q-learning 是**离线策略 (off-policy)** 的，而其他 TD 算法是在**线策略 (on-policy)** 的。

任何 RL 任务中都存在两个策略：**行为策略**（生成经验样本）和**目标策略**（不断更新以收敛到最优策略）。当行为策略与目标策略相同时，称为在线策略学习；不同时称为离线策略学习。

离线策略学习的优势在于可以基于其他策略生成的经验样本学习最优策略。行为策略可以选择为强探索性的，以提高学习效率。

判断方法：

- **Sarsa 是在线策略的。** Sarsa 先通过求解贝尔曼方程评估策略 π （需 π 生成的样本），再基于估计值改进策略。行为策略和目标策略相同。从样本生成看： $s_t \xrightarrow{\pi_b} a_t \xrightarrow{\text{model}} r_{t+1}, s_{t+1} \xrightarrow{\pi_b} a_{t+1}$ ，Sarsa 评估的策略正是生成样本的策略。
- **Q-learning 是离线策略的。** Q-learning 求解的是贝尔曼最优方程而非给定策略的贝尔曼方程。样本生成： $s_t \xrightarrow{\pi_b} a_t \xrightarrow{\text{model}} r_{t+1}, s_{t+1}$ 。行为策略 π_b 只用于生成 a_t ； (r_{t+1}, s_{t+1}) 的生成由环境决定，不涉及 π_b 。因此目标策略可以与行为策略不同。目标策略是基于估计的最优价值获得的贪心策略。
- **MC 学习是在线策略的。** 原因与 Sarsa 类似。

另一个需区分的概念是**在线 (online)** / **离线 (offline)**：在线学习指边交互边更新；离线学习指用预收集数据更新。在线策略算法只能以在线方式实现；离线策略算法两种方式均可。

7.4.3 实现

Q-learning 可以以在线策略或离线策略方式实现。

算法 7.2: 通过 Q-learning 学习最优策略（在线策略版本）

初始化: $\alpha_t(s, a) = \alpha > 0$ 对所有 (s, a) 和所有 t 。 $\epsilon \in (0, 1)$ 。 初始化 $q_0(s, a)$ 对所有 (s, a) 。 由 q_0 导出初始 ϵ -贪心策略 π_0 。

目标: 学习能从 s_0 引导智能体到达目标状态的最优路径。

对于每个回合:

1. 如果 s_t ($t = 0, 1, 2, \dots$) 不是目标状态:

- (a) 给定 s_t 收集经验样本 (a_t, r_{t+1}, s_{t+1}) : 遵循 $\pi_t(s_t)$ 生成 a_t ; 与环境交互生成 r_{t+1}, s_{t+1} 。
- (b) 更新 q 值:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - (r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)) \right]$$

- (c) 更新 s_t 处的 ϵ -贪心策略:

$$\pi_{t+1}(a | s_t) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s_t)|} (|\mathcal{A}(s_t)| - 1), & a = \arg \max_a q_{t+1}(s_t, a) \\ \frac{\epsilon}{|\mathcal{A}(s_t)|}, & \text{否则} \end{cases}$$

算法 7.3: 通过 Q-learning 学习最优策略（离线策略版本）

初始化: $q_0(s, a)$ 对所有 (s, a) 。 行为策略 $\pi_b(a | s)$ 对所有 (s, a) 。 $\alpha_t(s, a) = \alpha > 0$ 。

目标: 从 π_b 生成的经验样本中学习所有状态的最优目标策略 π_T 。

对于由 π_b 生成的每个回合 $\{s_0, a_0, r_1, s_1, a_1, r_2, \dots\}$:

1. 对于每步 $t = 0, 1, 2, \dots$:

- (a) 更新 q 值:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - (r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)) \right]$$

- (b) 更新目标策略为贪心:

$$\pi_{T,t+1}(a | s_t) = \begin{cases} 1, & a = \arg \max_a q_{t+1}(s_t, a) \\ 0, & \text{否则} \end{cases}$$

7.4.4 示例演示

第一个示例（图 7.3）演示了在线策略 Q-learning: 从起始状态找到到目标状态的最优路径。奖励 $r_{\text{target}} = 0$, $r_{\text{forbidden}} = r_{\text{boundary}} = -10$, $r_{\text{other}} = -1$, $\alpha = 0.1$, $\epsilon = 0.1$ 。Q-learning 最终找到最优路径，回合长度减少，总奖励增加。

第二个示例（图 7.4 和 7.5）演示了离线策略 Q-learning: 为所有状态找最优策略。 $r_{\text{boundary}} = r_{\text{forbidden}} = -1$, $r_{\text{target}} = 1$, $\gamma = 0.9$, $\alpha = 0.1$ 。真实值由基于模型的策略迭代获得（图 7.4(a,b)）。行为策略为均匀分布 ($\pi(a | s) = 0.2$)，生成 100,000 步的单一回合。学习到的策略（图 7.4(e)）是最优的，状态价值误差收敛到零（图 7.4(f)）。由于 Q-learning 是自助式的，初始猜测影响性能: 接近真实值时约 10,000 步收敛（图 7.4(g)），否则需要更多步（图 7.4(h)）。行为策略的探索能力影响显著: ϵ 从 1 降到 0.5 再到 0.1 时，学习速度显著下降（图 7.5）。

7.5 统一视角

目前为止，我们介绍了 Sarsa、 n 步 Sarsa 和 Q-learning。本节给出统一框架来容纳所有这些算法以及 MC 学习。

TD 算法（用于动作价值估计）可表示为统一表达式：

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)[q_t(s_t, a_t) - \bar{q}_t], \quad (6)$$

其中 $\bar{q}_t$ 是 **TD 目标**。不同算法的 TD 目标见表 2。MC 学习可视为 $\alpha_t = 1$ 的特例： $q_{t+1}(s_t, a_t) = \bar{q}_t$ 。

算法(6)可视为求解 $q(s, a) = \mathbb{E}[\bar{q}_t | s, a]$ 的随机逼近算法，该方程在不同 $\bar{q}_t$ 下有不同表达式（表 3）。除 Q-learning 求解贝尔曼最优方程外，所有算法都求解贝尔曼方程。

表 2: TD 算法的统一视角：TD 目标 $\bar{q}_t$ 的表达式

算法	$\bar{q}_t$ 的表达式
Sarsa	$\bar{q}_t = r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$
n 步 Sarsa	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_t(s_{t+n}, a_{t+n})$
Q-learning	$\bar{q}_t = r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)$
蒙特卡洛	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots$

表 3: 各算法旨在求解的方程（BE：贝尔曼方程，BOE：贝尔曼最优方程）

算法	待求解的方程
Sarsa	BE: $q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) S_t = s, A_t = a]$
n 步 Sarsa	BE: $q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_\pi(S_{t+n}, A_{t+n}) s, a]$
Q-learning	BOE: $q(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_a q(S_{t+1}, a) S_t = s, A_t = a]$
蒙特卡洛	BE: $q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots s, a]$

7.6 本章小结

本章介绍了一类重要的强化学习算法——TD 学习，包括 Sarsa、 n 步 Sarsa 和 Q-learning。所有这些算法都可视为求解贝尔曼方程或贝尔曼最优方程的随机逼近算法。

除 Q-learning 外，本章的 TD 算法用于评估给定策略（从经验样本中估计状态/动作价值），结合策略改进可学习最优策略。这些算法是在线策略的：目标策略用作行为策略生成经验样本。

Q-learning 是离线策略的，目标策略可与行为策略不同。根本原因是 Q-learning 旨在求解贝尔曼最优方程而非给定策略的贝尔曼方程。

将在线策略算法转换为离线策略算法可使用重要性采样 [3, 40]（第 10 章介绍）。TD 算法还有众多变体和扩展 [41-45]，如 TD(λ) 方法提供了更一般的统一框架 [3, 20, 46]。

7.7 常见问题解答

• Q: TD 学习中的“TD”是什么意思？

A: 每个 TD 算法都有一个 TD 误差，表示新样本与当前估计之间的差异。由于该差异在不同时间步之间计算，因此称为时序差分（temporal-difference）。

- **Q: TD 学习中的”学习”是什么意思?**
A: 从数学角度看, ”学习”就是”估计”——从样本中估计状态/动作价值, 然后基于估计价值获得策略。
- **Q: Sarsa 如何用于学习最优策略?**
A: 价值估计过程必须与策略改进过程交互 (广义策略迭代)。价值更新后立即更新策略, 新策略生成新样本用于继续估计。
- **Q: 为什么 Sarsa 使用 ϵ -贪心策略?**
A: 策略也用于生成估计价值的样本, 需要探索性以生成充足经验样本。
- **Q: 定理 7.1 和 7.2 要求学习率衰减到零, 为什么实践中常用常数?**
A: 在最优策略学习过程中, 被评估的策略不断变化 (非平稳)。常数学习率能有效跟踪变化。衰减学习率可能太小而无法有效评估。虽然常数学习率可能导致最终波动, 但只要足够小就可以忽略。
- **Q: 应学习所有状态还是部分状态的最优策略?**
A: 取决于任务。如寻找固定起始到目标的路径, 不需要探索所有状态, 但可能只获得局部最优路径。充分探索可以保证全局最优性。
- **Q: 为什么 Q-learning 是离线策略的而其他算法是在线策略的?**
A: 根本原因是 Q-learning 求解贝尔曼最优方程, 而其他算法求解给定策略的贝尔曼方程。详见第 7.4.2 节。
- **Q: 离线策略 Q-learning 为什么更新为贪心而非 ϵ -贪心策略?**
A: 目标策略不用于生成经验样本, 因此不需要具备探索性。